Yashwardhan Panchal
CSCI 5561 - Computer Vision
October 20, 2026

HW2 Report

Assignment item	Points	Location in report	Extra credit?
Report Quality	10	-	No
Two-Dimensional Flow Matching	15	Section 1, p. 2	No
Image Model and Training Diagnostics	20	Section 2, p. 4	No
Unconditional Image Generation	20	Section 3, p. 6	No
Basic Performance Tuning	10	Section 4, p. 7	No
Advanced Performance Tuning	20	Section 4, p. 7	No
Interactive Sampler	5	Section 5, p. 11	No
CIFAR-10	15	Appendix A, p. 12	Yes
Generation Animation	5	Appendix B, p. 14	Yes
Total attempted before extra-credit conversion	120		
Maximum possible score after conversion	110		

Introduction

In this project, I implement flow matching from scratch to train generative models on two-dimensional point distributions and image datasets. Starting from a 2D velocity field on a custom point distribution, I scale the method to image generation using U-Net architectures, evaluating one-batch overfitting diagnostics, full unconditional MNIST handwritten digit generation, and adaptation to CIFAR-10 color images. All models and experiments throughout this report were trained and evaluated on a local GPU.

1 Two-Dimensional Flow Matching

This version uses a simple 3-Layer MLP for flow matching. Given a data point x_1 and initial Gaussian noise $x_0 \sim \mathcal{N}(0, I)$, the model defines a linear probability path $x_t = (1-t)x_0 + tx_1$ for $t \in [0, 1]$ with target velocity $v_t = x_1 - x_0$. The network is trained using the flow-matching mean squared error loss:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t\sim U[0,1],x_0,x_1}\left[\|v_\theta(x_t, t) - (x_1 - x_0)\|^2\right] \tag{1}$$

The input layer takes in 3 values for each sample: x, y, t. Each hidden layer has a width of 128 and a ReLU activation between them. The last layer projects the 128 features into two outputs: velocity in the x and y directions.

The training uses the Adam optimizer, a learning rate of 0.001, the MSE loss function in Equation (1), and 4000 training steps with batch size 512 for each step. The final model has 33,794 weights and a loss of 1.3736.

Figure 1.1 shows the comparison between the True distribution and the one generated from the model. The model has successfully generated a similar distribution that resembles a dog face, with some smearing of points around the edges. For example, the ears and nose resemble the original distribution in a coarse way and lose some of the details and sharpness.

At $t = 0$, the velocities for all samples point towards the center or the mean of the distribution, as seen in Figure 1.2. Halfway through, at $t = 0.5$, the velocities of samples towards the edges are still pointing inwards, but those in the center are now pointing outwards. At this point, it slightly resembles the original distribution; the ears and mouth can be seen forming vaguely.

Finally, at $t = 1$, the velocities point towards the final destination. Samples in the center point to the closest facial feature.

Figure 1.3 shows the trajectories for 120 samples. Many samples initially overshoot their final destination before correcting their direction. This is likely due to the fact that the initial velocities for samples point towards the center of mass or mean of the distribution.

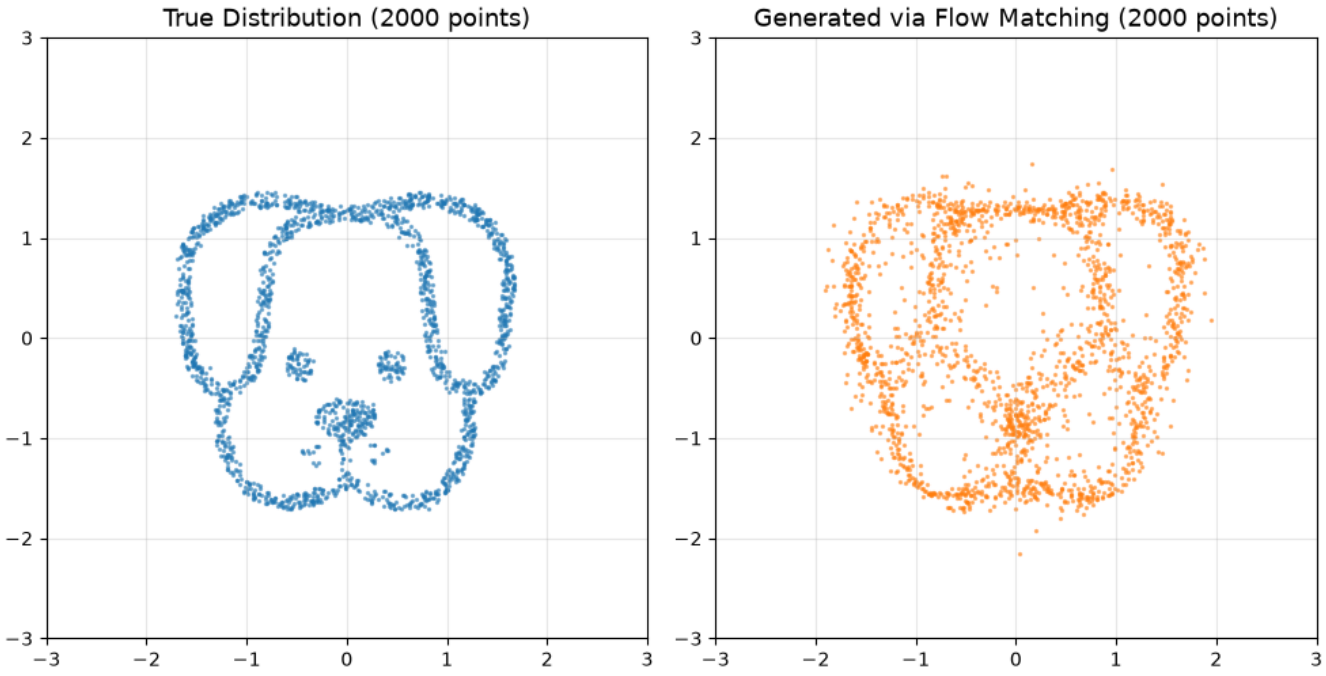


Figure 1.1 True vs Generated

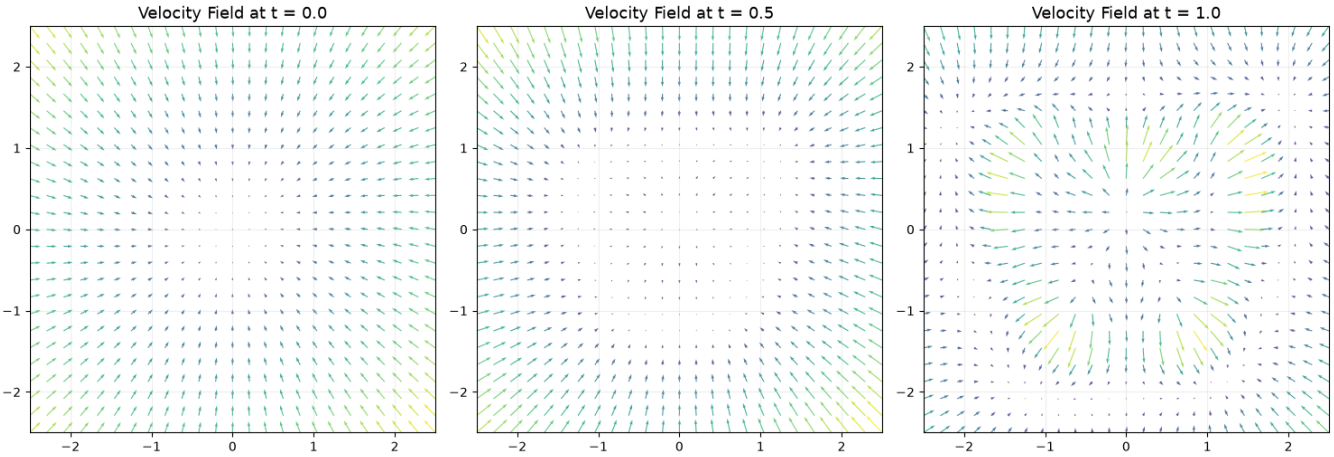


Figure 1.2 Velocity field at times $t = 0$, $t = 0.5$, and $t = 1$.

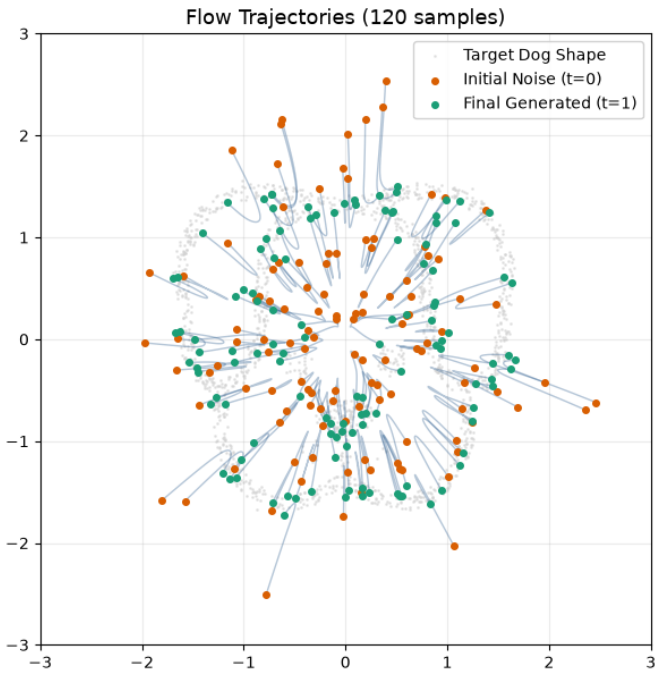


Figure 1.3 Flow trajectories from t=0 to t=1 for 120 samples

2 Image Model and Training Diagnostics

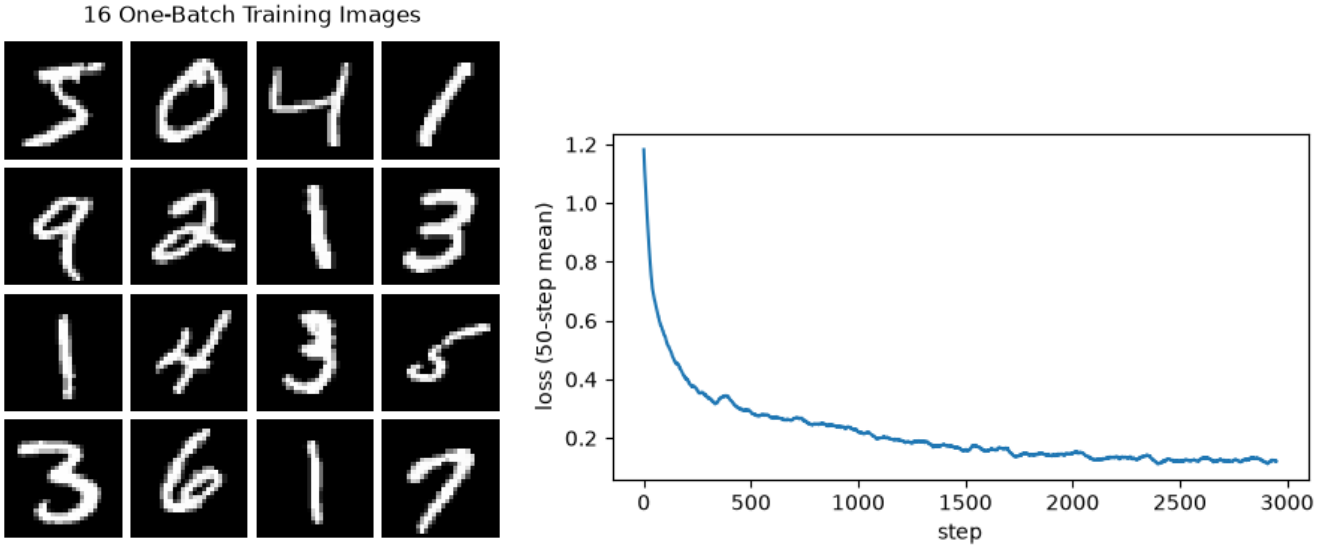
This section adapts flow matching to images using a small U-Net. The input is a noisy 28×28 grayscale image and a time value $t \in [0, 1]$. Time t is encoded using a 32-dimensional sinusoidal embedding followed by a two-layer MLP. The U-Net has two downsampling layers (widths 16 and 32) with 2×2 max pooling, a middle block with width 64, and two upsampling layers with skip connections to bring the resolution back to 28×28. A final 1×1 convolution outputs the predicted velocity for each pixel.

To test the model before training on the full dataset, it is trained to overfit a single batch of 16 MNIST images, shown in Figure 2.1. Image pixel values are scaled to $[-1, 1]$. The training uses the Adam optimizer with a learning rate of 0.001, MSE loss, batch size of 16, and 3000 steps. The model has 124,145 weights and trains in about 66 seconds on a GPU.

Figure 2.1 shows the loss curve over the 3000 steps. The loss drops quickly from around 1.2 to 0.3156 at step 500, and levels off around 0.1196. The loss does not reach 0 because random noise is generated every step. Near $t = 0$, the input is mostly pure noise, so the model cannot tell which of the 16 images it came from and can only guess the average direction.

Figure 2.2 shows 16 generated samples from fixed noise seeds using 50 Euler steps. At step

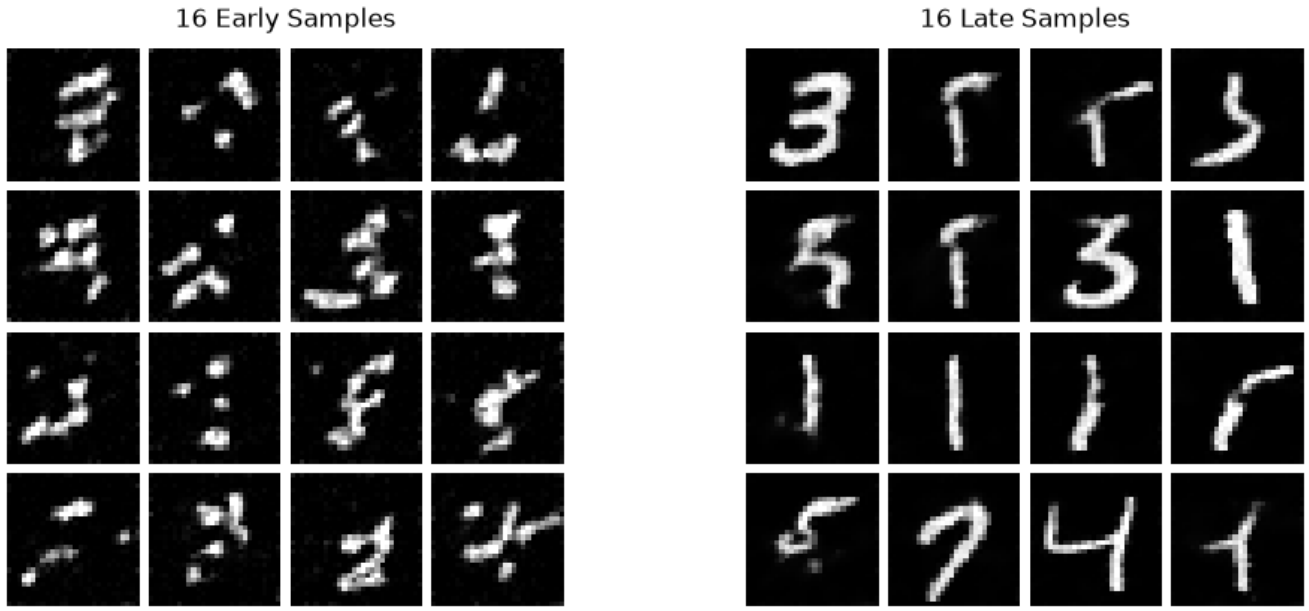
500 (Figure 2.2(a)), the samples are blurry and only show rough stroke shapes. By step 3000 (Figure 2.2(b)), the images become sharp, clear digits that match the training batch, especially the 1s and 3s. Digits like 1 and 3 appear more often because they make up almost half of the 16 training images. This confirms that the model can properly memorize and generate image data before moving to full training.



(a) The 16 MNIST training images used for the one-batch overfitting diagnostic

(b) Training loss curve over 3,000 steps

Figure 2.1 One-batch overfitting diagnostic



(a) Early samples at step 500

(b) Late samples at step 3,000

Figure 2.2 Fixed-noise generated samples from 50 Euler steps at early and late stages of training

3 Unconditional Image Generation

After verifying that the one-batch model in Section 2 could learn and generate digits, the next step is scaling up to train on the entire MNIST dataset of 60,000 images. A larger model is needed to capture all the different handwriting variations across every digit class. Figure 3.1 shows the final architecture used for unconditional generation.

The final model expands the U-Net from Section 2 in several ways. The base channel width is increased from 16 to 64, with channel counts growing to 128 in the second downsampling level and 256 at the middle bottleneck. Each level now contains two residual blocks instead of standard convolutions. Inside each block, the features pass through a 3×3 convolution, GroupNorm with 4 groups, and have the 32-dimensional timestep embedding added after a linear projection. The features then pass through a SiLU activation and a second 3×3 convolution, which is initialized with zero weights, allowing the block to start as an identity shortcut before learning adjustments. Transposed convolutions and horizontal concatenation skip connections mirror the encoder to bring the features back to 28×28, where a 1×1 convolution predicts the final velocity. In total, this architecture has 3,909,441 trainable weights.

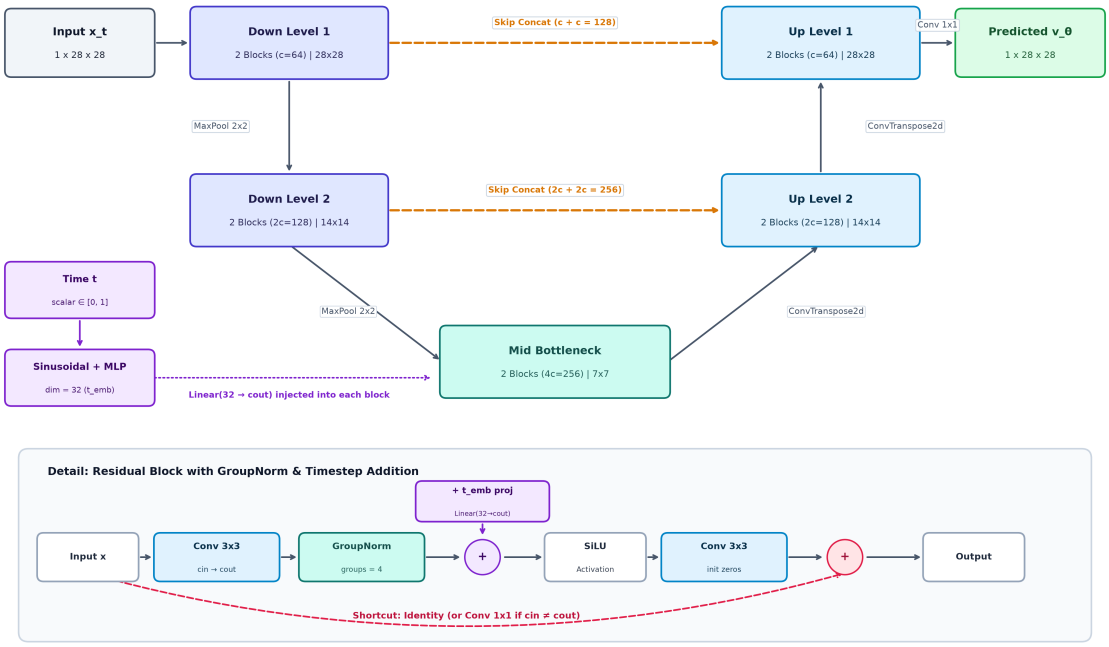


Figure 3.1 Architecture diagram of the final two-block Residual GroupNorm U-Net (width $c = 64$, 3.91M parameters).

4 Performance Tuning and Progression

To find the best configuration, I ran a series of 18 training trials testing different architectural changes, sampling methods, and additional strategies. Table 1 tracks the progress across these trials, showing the final loss and the classification recognition score evaluated with the pre-trained MNIST classifier. All trials below use Adam optimizer and a base learning rate of 0.001

The baseline model (Trial 1) from Section 2 used a simple U-Net with width 16 for 30 epochs, reaching a loss of 0.1982 and a 60.1% recognition score. Adding two blocks per level and residual connections with GroupNorm (Trials 2–5) improved the score to around 70.7%. The biggest reduction in loss came in Trial 7 with the introduction of Optimal Transport (OT) flow matching. Instead of pairing random noise with random training images, OT pairs points with similar locations, creating straighter flow trajectories. This dropped the training loss sharply from 0.1808 to 0.1291.

Adding a Cosine Learning Rate (CLR) schedule and testing a second-order Heun sampler (Trials 9–10) further brought the score to 76.9%. Scaling the network capacity made another big difference: increasing the base width to 32 (Trial 11) raised the score to 84.5%, and increasing to width 64 (Trial 16) reached 85.4%. I tried several other strategies with the width-32 version, such as 7×7 Attention, EMA, Beta t, and FiLM. Each of these was tested independently in a 30-epoch pilot run. Based on these tests, none of these modifications yielded significant performance improvements, showing very little variation in recognition score.

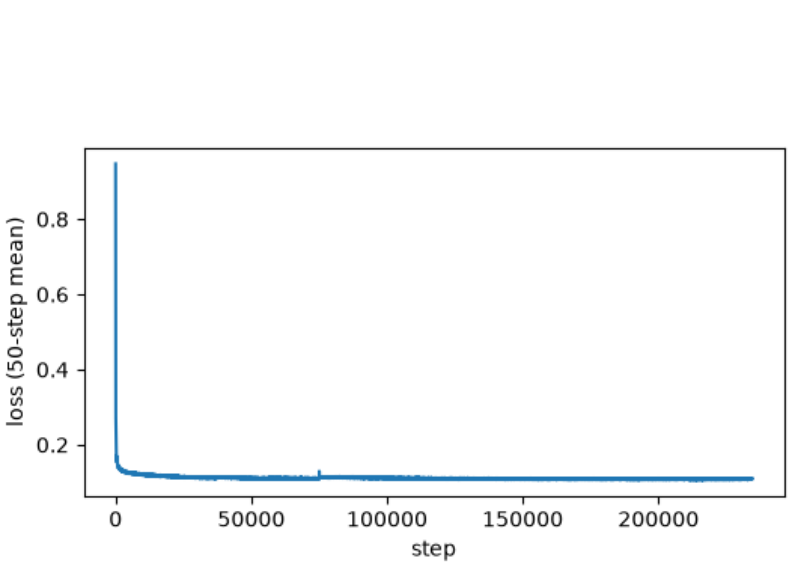
Finally, combining width 64 with GroupNorm, SiLU activations, batch size 128, and Euler sampling with 50 steps reached 86.5% at 80 epochs (Trial 18). Resuming training of this checkpoint to 120 and then to a total of 500 epochs with 100 Heun steps achieved the best result: a final loss of 0.1093 and a recognition score of 89.1% with at least 140 instances of each digit, ensuring full coverage. This took roughly 33 seconds per epoch, which is about 4.5 hours total on a single GPU.

Figure 4.1(a) shows the training loss curve for the final 500-epoch model. The loss decreases very slowly through all 500 epochs and levels out near 0.1093 without showing signs of overfitting. Figure 4.1(b) shows generated digits from the final model using 100 Heun steps. The generated images are sharp, clean, and show realistic handwritten digits across all numbers from 0 to 9 without

noticeable noise or blurriness. In Figure 4.1(b), samples with classifier confidence at or below 0.99 are highlighted with red borders. Some digits are fairly recognizable by humans but fail to be recognized by the classifier. For example, the digit in row 2, column 8 would be classified as a four with long strokes, but is misclassified due to being ambiguous.

Figure 4.2 shows the progression in quality through training. Though the samples appear fairly similar, observable improvements exist. Taking row 2, column 8 as an example again, it progresses from being a poorly drawn nine to a fairly recognizable four.

For comparison, Figure 4.3 shows samples from two earlier checkpoints. In Trial 7 (Figure 4.3(a)), the model could generate rough shapes, but some digits are broken or deformed (70.6% recognition score). By Trial 16 (Figure 4.3(b)), the shapes are much better, but some strokes are still uneven.



(a) Final model loss curve across 500 epochs



(b) Final model generated samples

Figure 4.1 Training loss curve and generated samples from the final model (Trial 18, 500 epochs).

Description	Loss	Recognition Score
Trial 1: Baseline (30 epochs)	0.1982	60.1%
Trial 2: Two blocks per level (30 epochs)	0.1854	70.1%
Trial 3: Two blocks per level + GroupNorm (30 epochs)	0.1850	69.2%
Trial 4: Two blocks per level + Residual (30 epochs)	0.1818	70.5%
Trial 5: Two blocks per level + Residual + GroupNorm (30 epochs)	0.1808	70.7%
Trial 6: Two blocks per level + Residual + GroupNorm (100 epochs)	0.1757	73.8%
Trial 7: Two blocks per level + Residual + OT (30 epochs)	0.1291	70.6%
Trial 8: Two blocks per level + Residual + OT (60 epochs)	0.1270	74.8%
Trial 9: Two blocks per level + Residual + OT + Heun (60 epochs)	0.1282	75.6%
Trial 10: Two blocks per level + Residual + OT + Heun + CLR (60 epochs)	0.1230	76.9%
Trial 11: Two blocks + Residual + OT + Heun + CLR (width 32, 60 epochs)	0.1159	84.5%
Trial 12: Two blocks + Residual + OT + Heun + CLR + Attention (width 32, 60 epochs)	0.1153	82.3%
Trial 13: Two blocks + Residual + OT + Heun + CLR + EMA (width 32, 60 epochs)	0.1158	84.0%
Trial 14: Two blocks + Residual + OT + Heun + CLR + Beta t (width 32, 60 epochs)	0.1202	84.1%
Trial 15: Two blocks + Residual + OT + Heun + CLR + FiLM (width 32, 60 epochs)	0.1140	83.8%
Trial 16: Two blocks + Residual + OT + Heun + CLR (width 64, 60 epochs)	0.1136	85.4%
Trial 17: Two blocks + Residual + OT + Heun + CLR + GroupNorm (width 64, SiLU, $B = 512$, $N = 100$, 80 epochs)	0.1036	82.9%
Trial 18: Two blocks + Residual + OT + Heun + CLR + GroupNorm (width 64, SiLU, $B = 128$, $N = 50$, 80 epochs)	0.1115	86.5%
Trial 18 (cont.): Resumed training to 120 epochs	0.1109	87.1%
Trial 18 (cont.): Resumed training to 160 epochs	0.1106	87.2%
Trial 18 (cont.): Resumed training to 500 epochs	0.1093	89.1%

Table 1 Summary of experimental trials and tuning progression on MNIST.



(a) Samples from epoch 80



(b) Samples from epoch 160

Figure 4.2 Quality progression along training for the final model



(a) Trial 7 samples (30 epochs, 70.6% score)



(b) Trial 16 samples (60 epochs, 85.4% score)

Figure 4.3 Generated samples from earlier trials showing the progression in quality.

5 Interactive Sampler

I implemented an interactive sampler application using Gradio. The trained U-Net velocity model weights and the pretrained course evaluation classifier are loaded once into GPU/CPU memory during startup. The UI features interactive sliders and selectors for:

- Random Seed: Controls the pseudo-random generator state for initial noise x_0.
- Step Count (N): Sets the number of sampler integration steps from $t = 0$ to $t = 1$.
- Sampling Method: Supports two different sampler methods (Euler, Heun).
- Noise: Noise temperature scale $\sigma \in [0.1, 2.5]$.
- Grid Size: Number of display samples (1 to 64 samples).
- Diagnostics: Highlights low-confidence samples, and shows predicted digit labels and flow trajectories.

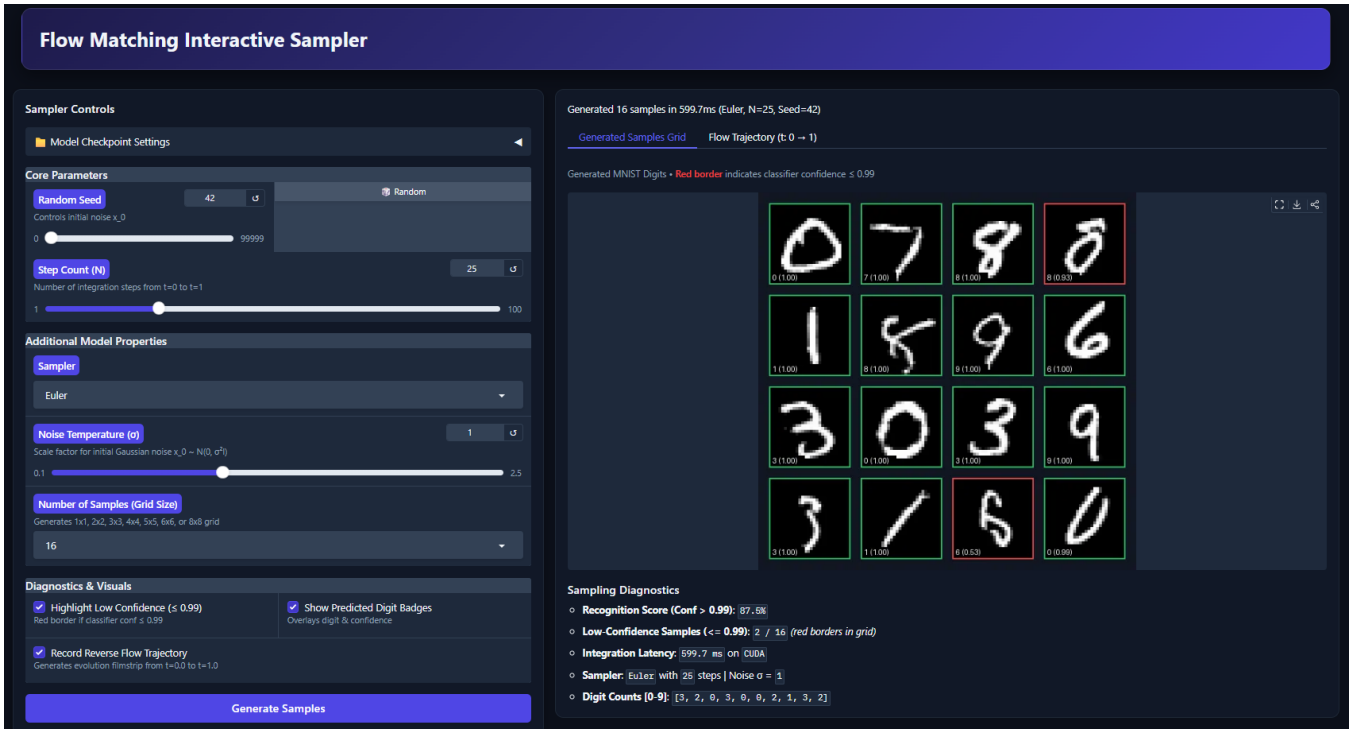


Figure 5.1 Display grid of samples

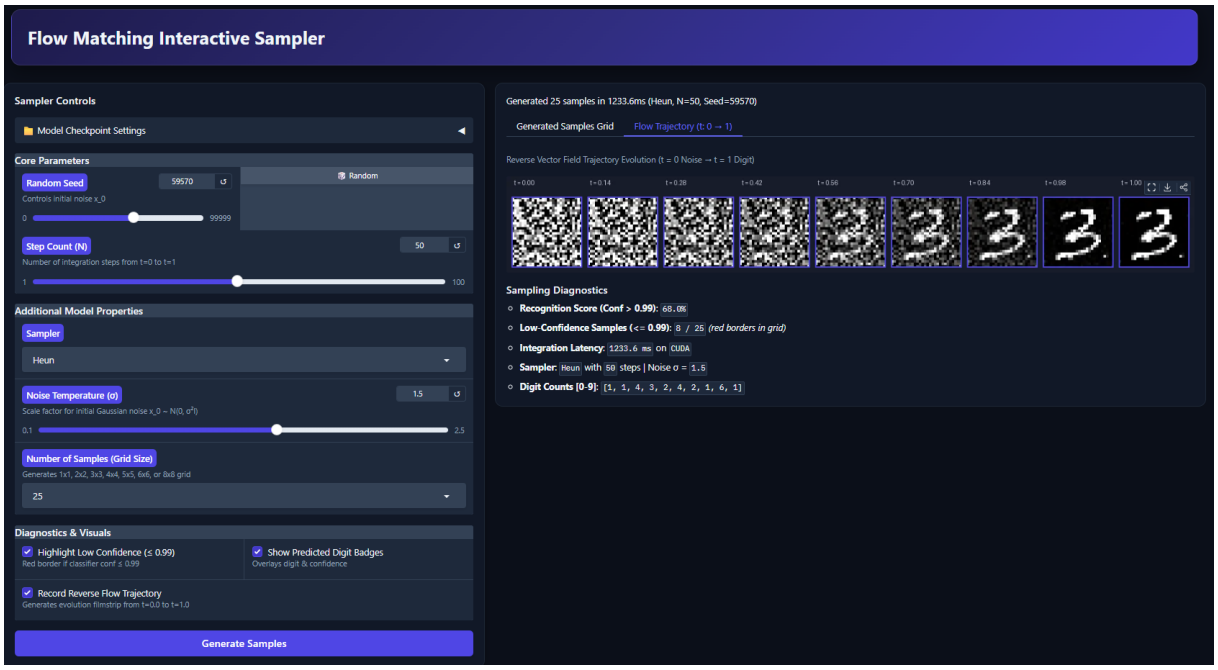


Figure 5.2 Showing flow evolution

Appendix A. CIFAR-10 Adaptation

After training the model on MNIST, I adapted the flow-matching pipeline to CIFAR-10 to test how well it performs on natural color images. CIFAR-10 contains 50,000 training images across 10 classes, with 3 RGB color channels at 32×32 resolution. All image pixels are scaled to $[-1, 1]$. No data augmentation was used, keeping the training objective simple and directly comparable to the MNIST setup.

The model uses the same two-block Residual GroupNorm U-Net architecture from Section 3 with base width 64. Only four minor changes were needed to handle CIFAR-10:

1. Input channels: The first level takes 3 RGB channels instead of 1 grayscale channel.
2. Output channels: The final 1x1 convolution outputs 3 channels to predict velocity for each color channel.
3. Spatial size: The feature maps start at 32×32, downsample to 16×16, and reach 8×8 at the middle bottleneck.
4. Model capacity: The network has 3,910,851 trainable weights, which is only 1,410 more parameters than the MNIST model (3,909,441).

The training setup follows the best configuration found in Section 4. Training ran for a total of 300 epochs with a batch size of 128 using the Adam optimizer with a base learning rate of 0.001.

A cosine learning rate schedule warms up linearly over the first 200 steps and decays to 0 over the remaining steps (117,300 total steps). Training on a GPU took about 33 seconds per epoch, totaling about 2.8 hours ($\sim$ 10,000 seconds). The loss started at 0.1966 at epoch 1, dropped steadily to 0.1669 by epoch 10, and leveled off at 0.1527 by epoch 300.

To generate images, I used the second-order Heun sampler with 50 integration steps. Figure A.1 shows an 8x8 grid of 64 generated CIFAR-10 samples from fixed random noise. The model produces realistic color palettes, showing natural backgrounds like blue skies, green grass, and dark asphalt roads. Several object classes are fairly recognizable, including cars, horses, birds, and ships. Because CIFAR-10 contains complex natural scenes with varied 3D viewpoints and textures compared to MNIST, some fine details and edges are soft without class conditioning. However, the overall spatial layouts and color transitions are coherent across all 64 samples. This shows that the flow-matching formulation and residual U-Net generalize well to 3-channel natural images with almost no architectural changes.

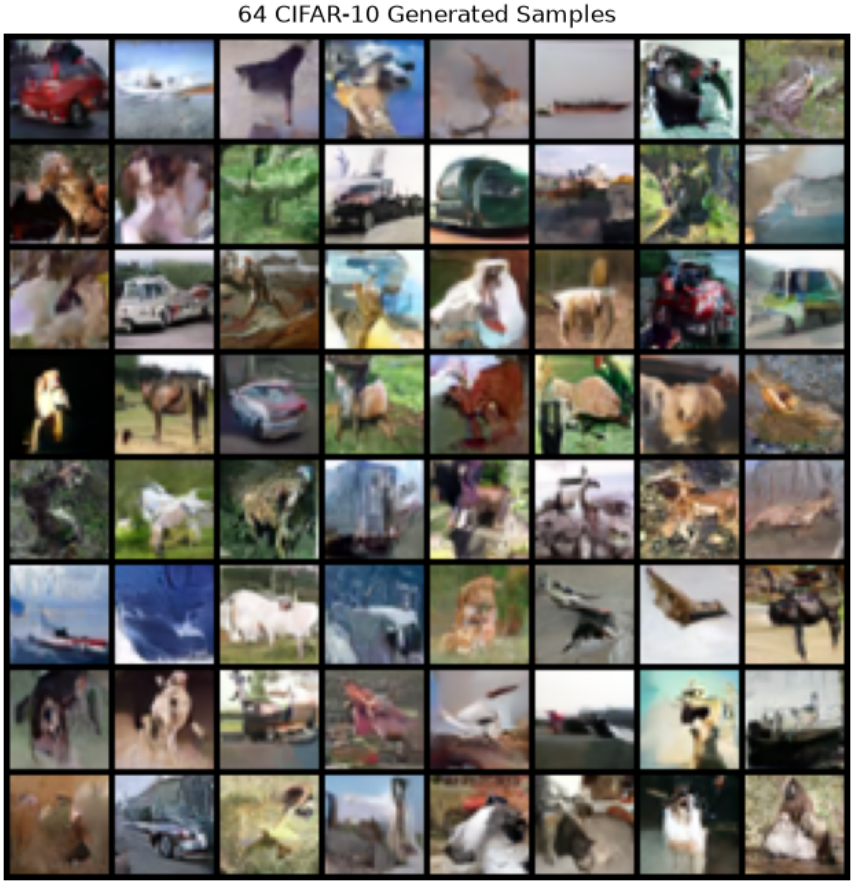


Figure A.1 64 generated CIFAR-10 samples from 50 Heun steps using the adapted width 64 Residual GroupNorm U-Net.

Appendix B. Generation Animation

To see how images form over time, I created an animation tracking CIFAR-10 samples evolving from random noise at $t = 0.0$ to final clean images at $t = 1.0$ using the second-order Heun solver with 50 steps. Figure B.1 shows a representative still with four samples at six different timesteps from $t = 0.0$ to $t = 1.0$. Watching the animation reveals how the model actually generates images step by step, which cannot be seen from static final images:

- Background colors appear first ($t \in [0.0, 0.2]$): The noise quickly smooths out, and general background colors like blue skies or green grass appear before any object shapes form.
- Rough shapes form in the middle ($t \in [0.3, 0.6]$): The model starts forming the rough outline and position of the object, such as the body of a car or a bird.
- Fine details sharpen at the end ($t \in [0.7, 1.0]$): Sharper edges and smaller details, like wheels or outlines, only appear in the final steps once the main shape is in place.
- Non-linear path: The animation shows that the process is not just a simple fade. The model quickly moves the noise toward a specific object type early on, and then slows down near the end to refine the details.

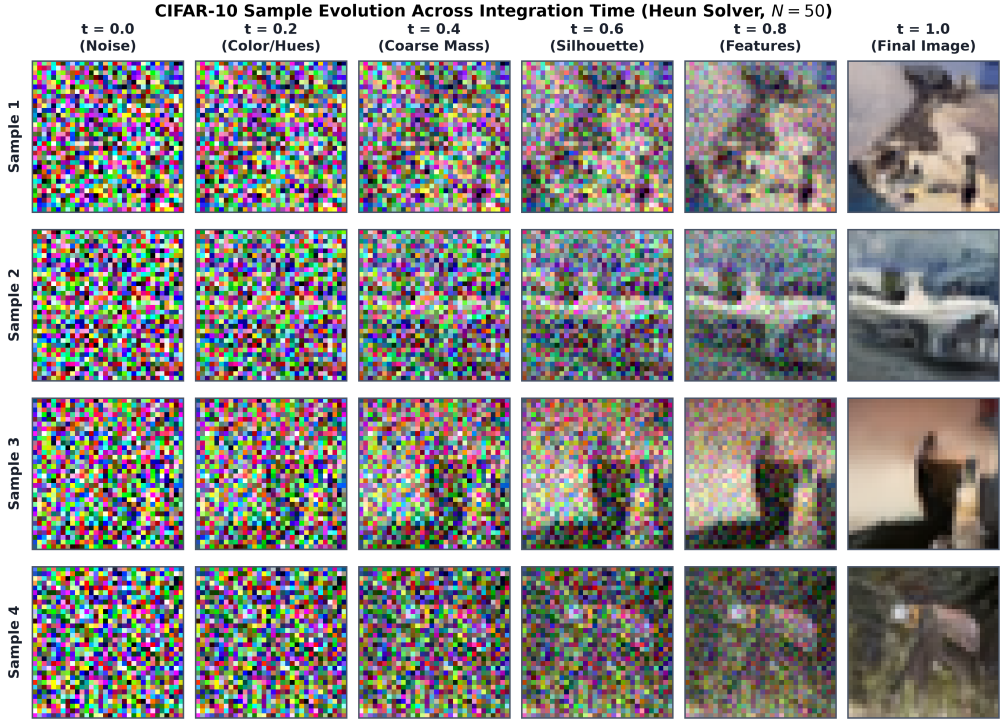


Figure B.1 Representative filmstrip from the CIFAR-10 generation animation showing sample progression across six integration timesteps ($t = 0.0$ to $t = 1.0$) using 50 Heun steps.